Version Control with Git, Part 2

Ben Winjum
OARC
January 29, 2026

Git + collaboration (using GitHub as example)

- Git already includes the machinery to move work between two repositories
- In practice, a central repository is usually used as a master copy
  - GitHub, BitBucket, GitLab…
- These hosting services also offer tools to facilitate collaboration
  - Web-based – anyone with an internet connection can view the repo
  - Wikis, task management, bug tracking, feature requests

Git + collaboration (using GitHub as example)

- Let’s make a new repository on GitHub

Git + collaboration (using GitHub as example)

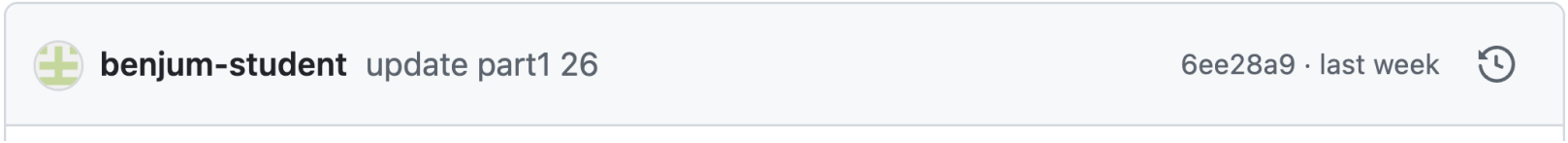
- Let’s make a new repository on GitHub
- After making it, we need to link our local repository with the remote repository
  - We’ll use HTTPS – you are encouraged to investigate configuring the SSH option later
  - `git remote add origin https://github.com/username/reponame.git`

Git + collaboration (using GitHub as example)

- Exchanging changes between repositories
  - `git push`
  - `git push origin main`
  - `git push –u origin main`
  - `git pull`
  - `git pull origin main`

GitHub GUI

- Indicators about repo updates on GitHub are spread across the page:



- Clicking on the clock, we find buttons like:



- Hex IDs; copying the commit code; browsing files in history

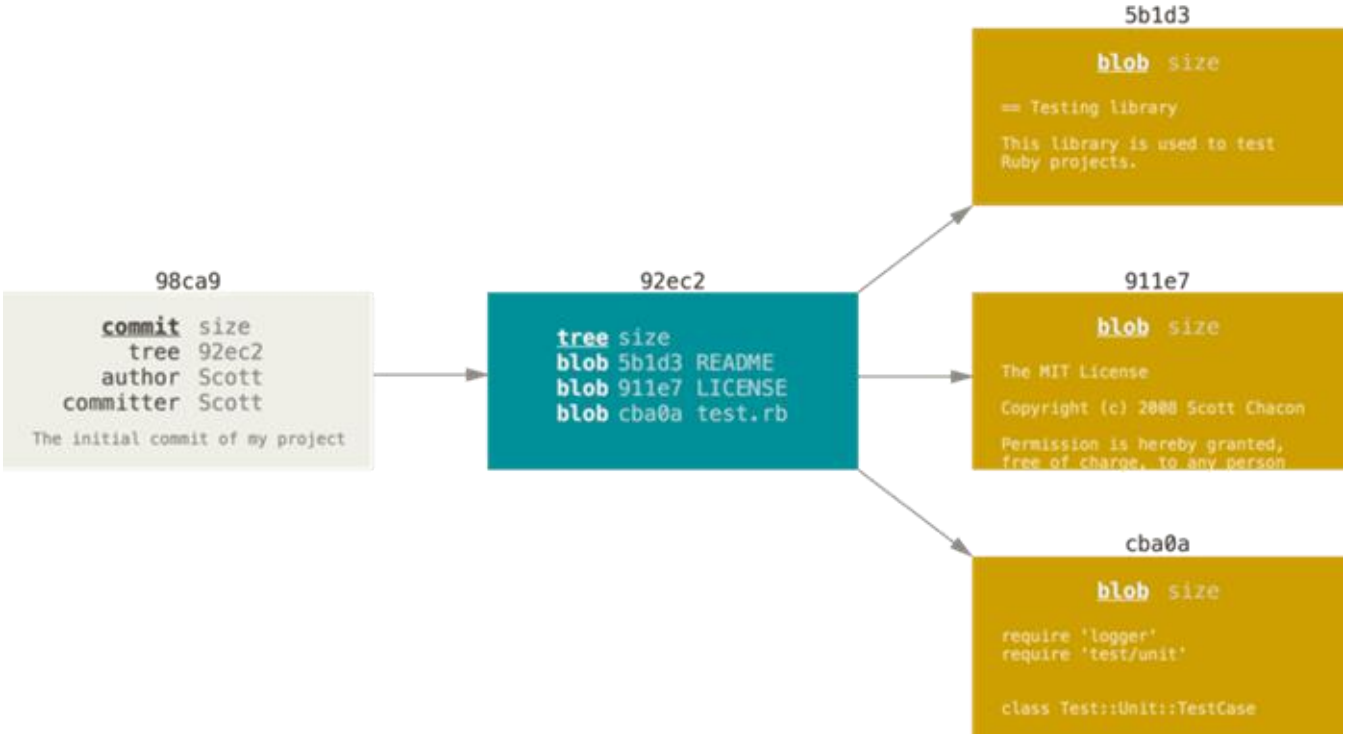
Collaborative workflow

- The basic collaborative workflow:
  - update your local repo with `git pull origin main`
  - make your changes and stage them with `git add`
  - commit your changes with `git commit -m`
  - upload the changes to GitHub with `git push origin main`
- It is better to make many commits with smaller changes rather than one massive commit with lots of changes
  - Small commits are easier to read and review

Branching

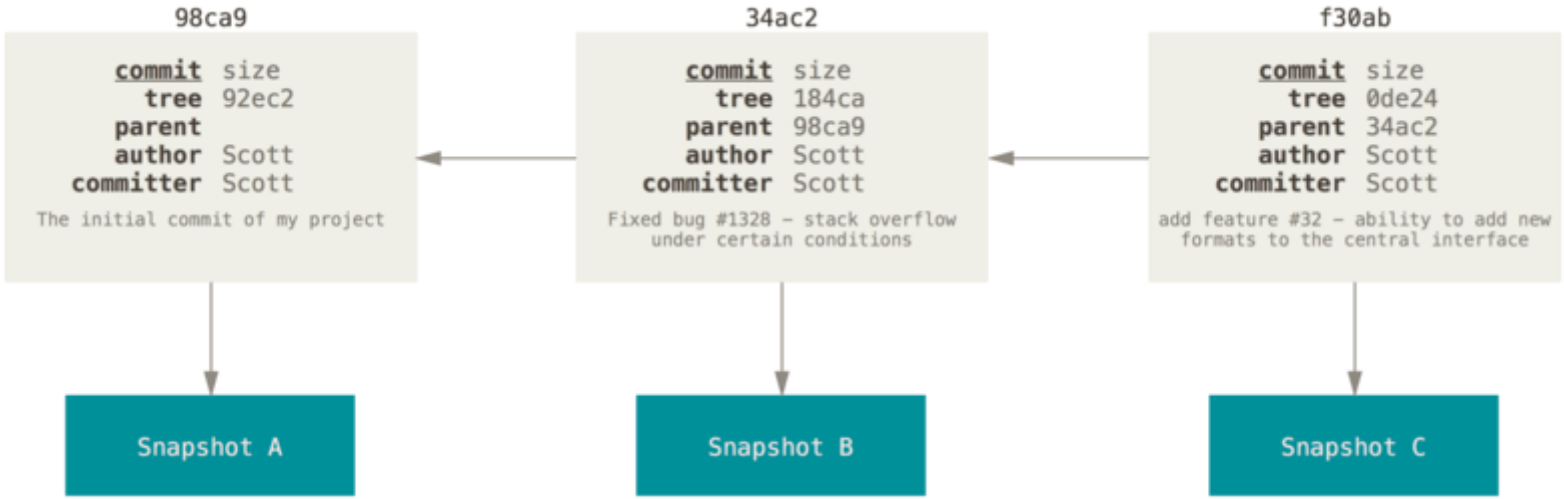
- Branching is one of the key features of git
- Git has a very light-weight method for developers to work simultaneously and separately on a project (in separate branches) and then merge their work together when they are ready
- To understand this, it helps to visualize some of the conceptual structure behind git

Files and the commit history



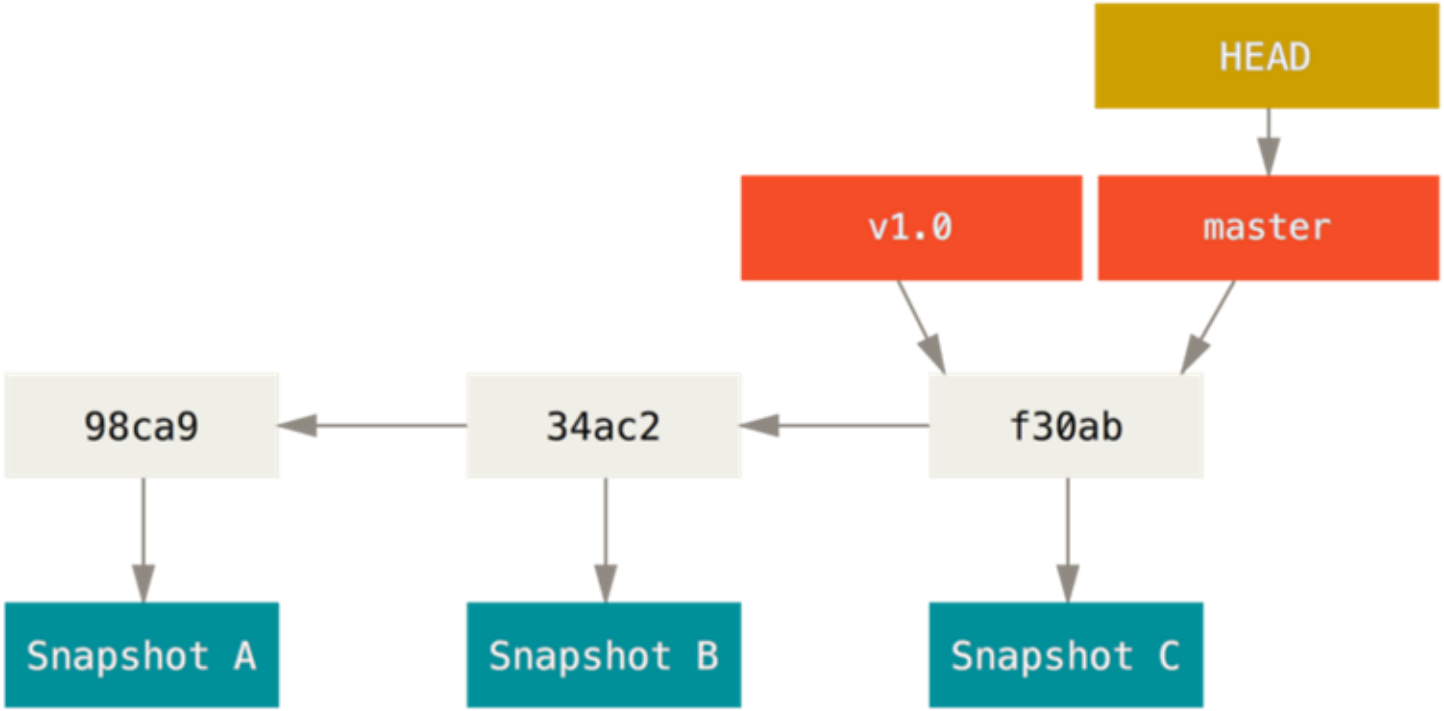
Images taken from the Pro Git book -- freely available online and recommended for further reading
https://git-scm.com/book/en/v2

Files and the commit history



Images taken from the Pro Git book -- freely available online and recommended for further reading
https://git-scm.com/book/en/v2

Pointers to commits



Images taken from the Pro Git book -- freely available online and recommended for further reading
https://git-scm.com/book/en/v2

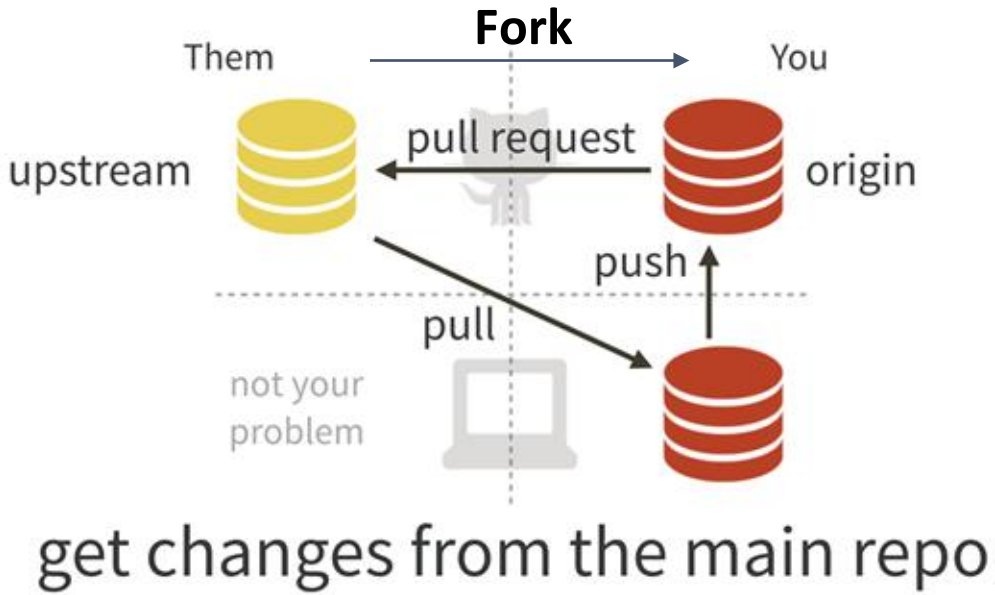
Branching commands

- Create a new branch
  - git branch <branchname>
- Switch to a branch
  - git checkout <branchname>
- Create a new branch and switch to it at the same time
  - git checkout -b <branchname>
- Delete a branch
  - git branch –d <branchname>

Branching example

- Create a new file for elementary math examples (with an error)
  - Commit to main
- Add features to the math file
  - Create a branch for this development path
- Correct the original error
  - Create a hotfix branch, fix the error, merge into main, delete hotfix branch
- Finish editing
  - Try merging math into main --> pull request illustration

Forks, Pushes, and Pull Requests



https://happygitwithr.com/upstream-changes.html

Some final thoughts: If you run into lots of conflicts

- Conflict resolution costs time and effort and can introduce errors if conflicts are not resolved correctly. If you find yourself resolving a lot of conflicts, consider these technical approaches:
  - Pull from upstream more frequently, especially before starting new work
  - Use topic branches to segregate work, merging to main when complete
  - Make smaller more atomic commits
  - Where logically appropriate, break large files into smaller ones so that it is less likely that two authors will alter the same file simultaneously
- Conflicts can also be minimized with project management strategies:
  - Clarify who is responsible for what areas with your collaborators
  - Discuss what order tasks should be carried out in with your collaborators so that tasks expected to change the same lines won't be worked on simultaneously
  - If the conflicts are stylistic churn (e.g. tabs vs. spaces), establish a project convention that is governing and use code style tools (e.g. htmltidy, perltidy, rubocop, etc.) to enforce, if necessary

Final item:
please complete the brief
post-course survey

Any Future Questions:
Email – bwinjum@oarc.ucla.edu